REPORTE TÉCNICO: TAREA 2.2

Transición de Diagrama E-R a Notación UML para POO

1. Introducción y Propósito de la Transición Conceptual

El presente reporte técnico documenta la transición formal del modelo conceptual Entidad-Relación (E-R / E-RE) desarrollado en la Tarea 2.1 hacia un **Diagrama de Clases en notación UML** para la empresa 'TechMarket Express'. Mientras que el modelo E-R enfoca su propósito en la estructura estática y persistencia de los datos en un Sistema Gestor de Bases de Datos (SGBD), el diagrama de clases UML integra la visión de desarrollo orientado a objetos (POO), combinando atributos, comportamientos (métodos), encapsulamiento y relaciones de software.

2. Matriz Comparativa Detallada: E-R vs. Notación UML

En la siguiente matriz se establece la correspondencia directa elemento por elemento entre ambos paradigmas de diseño:

Concepto Modelo E-R	Notación UML (Clases)	Descripción y Reglas de Transición
Entidad (Fuerte / Débil)	Clase	Las entidades se convierten en Clases. Sintaxis en singular y PascalCase (ej. `Cliente`, `Pedido`).
Atributo Simple	Propiedad / Atributo de Clase	Atributos pasan a ser propiedades con visibilidad explícita (`-` privado) y tipo de dato (ej. `- rfc: String`).
Atributo Clave Primaria (PK)	Identificador / Estereotipo	Se indica mediante el estereotipo `<>` o la restricción `{id}` en la propiedad.
Atributo Compuesto	Clase de Soporte / Aplanado	Se descompone en propiedades independientes dentro de la clase o en una clase de soporte (ej. `Direccion`).
Atributo Multivalorado	Colección / Lista (`List`)	Se modela como un atributo de tipo lista/colección (ej. `- telefonos: List`) o una relación 1..*.
Atributo Derivado	Propiedad Calculada / Método	Se representa como propiedad calculada con prefijo `/` o mediante un método getter (ej. `+ getEdad(): int`).
Relación / Interrelación	Asociación Binaria	Las relaciones se convierten en Asociaciones con roles en los extremos y navegabilidad definida.
Cardinalidad (1:1, 1:N, N:M)	Multiplicidad	1:1 -> `1..1`, 1:N -> `1..*` o `0..*`, N:M -> `*..*` (esta última requiere Clase de Asociación).
Agregación (E-R Extendido)	Agregación / Composición UML	Distinción de ciclo de vida: Composición (`◆` destrucción dependiente) o Agregación (`■` independiente).
Especialización / Generalización	Herencia / Generalización	Relación 'Es-Un' representada por la flecha con triángulo hueco dirigida hacia la superclase.

3. Análisis de Coincidencias e Impacto en Programación Orientada a Objetos

Encapsulamiento y Visibilidad: En el modelo E-R todos los datos son accesibles públicamente mediante consultas SQL. En UML, se aplica el principio de ocultamiento de información: los atributos se declaran privados (`-`) y el acceso se gestiona mediante métodos públicos (`+`) getters y setters.

Inclusión de Comportamiento (Métodos de Negocio): La principal diferencia radica en que las clases UML incorporan métodos operacionales que ejecutan las reglas de negocio, como `+ calcularDescuentoVIP()`, `+ registrarPedido()`, `+ validarStock()`, unificando estado y comportamiento.

Mapeo Objeto-Relacional (ORM): Esta transición conceptual facilita la implementación técnica utilizando marcos de persistencia de código como Hibernate, Entity Framework o SQLAlchemy, donde cada clase UML se mapea a una tabla relacional y cada objeto a un registro.

4. Mapeo Formal de Clases UML (Ejemplo Completo de Código Conceptual)

A continuación se presenta la representación textual formal en UML de las clases centrales `Cliente` y `Pedido`:

```
+------------------------------------------------------------+
| Cliente |
+------------------------------------------------------------+
| - idCliente: int {id} |
| - rfc: String {unique} |
| - nombreCompleto: String |
| - email: String {unique} |
| - fechaNacimiento: Date |
| - telefonos: List<String> |
+------------------------------------------------------------+
| + getIdCliente(): int |
| + getEdad(): int {query / calculada} |
| + agregarTelefono(numero: String): void |
| + realizarPedido(p: Pedido): void |
+------------------------------------------------------------+
```

```
+------------------------------------------------------------+
| Pedido |
+------------------------------------------------------------+
| - idPedido: int {id} |
| - fechaPedido: DateTime |
| - estado: String |
| - cliente: Cliente |
| - detalles: List<DetallePedido> |
+------------------------------------------------------------+
| + calcularMontoTotal(): double {query} |
| + cambiarEstado(nuevoEstado: String): boolean |
| + procesarPago(pago: Pago): boolean |
+------------------------------------------------------------+
```

5. Conclusiones y Referencias APA

La conversión del modelo conceptual E-R a un diagrama de clases UML constituye un paso fundamental para conectar el diseño de la base de datos con la arquitectura de software. La notación UML proporciona una visión integral que abarca tanto la persistencia de datos como la lógica de negocio, garantizando un desarrollo modular, mantenible y orientado a objetos.

Referencias Bibliográficas (Formato APA):

• Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.

• Larman, C. (2005). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design* (3rd ed.). Prentice Hall.